

A memetic algorithm to solve an unrelated parallel machine scheduling problem with auxiliary resources in semiconductor manufacturing

Abdoul Bitar · Stéphane Dauzère-Pérès ·
Claude Yugma · Renaud Roussel

Received: 30 November 2013 / Accepted: 6 September 2014 / Published online: 2 October 2014
© Springer Science+Business Media New York 2014

Abstract In this paper, we propose a metaheuristic for solving an original scheduling problem with auxiliary resources in a photolithography workshop of a semiconductor plant. The photolithography workshop is often a bottleneck, and improving scheduling decisions in this workshop can help to improve indicators of the whole plant. Two optimization criteria are separately considered: the weighted flow time (to minimize) and the number of products that are processed (to maximize). After stating the problem and giving some properties on the solution space, we show how these properties help us to efficiently solve the problem with the proposed memetic algorithm, which has been implemented and tested on large generated instances. Numerical experiments show that good solutions are obtained within a reasonable computational time.

Keywords Scheduling · Parallel machines · Setups · Auxiliary resources · Semiconductor Manufacturing · Memetic algorithm

1 Introduction

Semiconductor manufacturing consists of producing integrated circuits that can be found in everyday electronic devices. The fabrication can be described as a multiple-step sequence of operations on a wafer made of semiconducting material, generally silicon. Typical processing steps are chemical mechanical polishing, diffusion, film deposition, photolithography, implant (doping), or etching. For each of the product types, and depending on the technology, a wafer goes through more than 400 process steps over a period of a few weeks. Wafer scheduling is a complex task due to the large number of products and machines involved. It is further complicated by additional constraints such as, for example, the reentrant flow of operations, time constraints to avoid contamination, setup issues, and preventive maintenance. The importance of scheduling in the performance of semiconductor wafer fabrication facilities (fabs) has been known for many years [Mönch et al. \(2011, 2012\)](#).

In this paper, we focus on the photolithography workshop, which is generally a bottleneck area. Lots of *wafers*, called *jobs* in this paper, must be processed on nonidentical parallel machines. A lot has at most 25 wafers. The processing of a lot in photolithography consists in transferring an integrated circuit pattern on a wafer. To perform this operation, some resin is placed on the wafer, which is exposed to an ultraviolet ray. At this step, the use of an auxiliary resource, i.e., a specific *mask* (also called a *reticle*), is necessary to shape the pattern on the wafer. The machine can only start processing when both the job and the right reticle are available. Since the reticle must be on the machine for the duration of the process, the relatively small number of reticles present in a wafer fab further complicates the parallel machine scheduling problem. Indeed, because of their cost, there is only one mask of each type. Moreover, machines need a specific configura-

A. Bitar (✉) · S. Dauzère-Pérès · C. Yugma
Ecole Nationale Supérieure des Mines de Saint-Etienne,
CMP Georges Charpak, Department of Manufacturing Sciences
and Logistics, CNRS UMR 6158 LIMOS, Center
of Microelectronics in Provence, 13541 Gardanne, France
e-mail: Bitar@emse.fr

S. Dauzère-Pérès
e-mail: Dauzere-Peres@emse.fr

C. Yugma
e-mail: Yugma@emse.fr

R. Roussel
STMicroelectronics Crolles, 38926 Crolles, France
e-mail: Renaud.Roussel@st.com

tion, called a *batch configuration*, to process a lot; switching from one batch configuration to another requires a setup time, depending on the previous job, the next job, and the machine that will process both jobs. Finally, machines are *qualified* (or eligible) only for a set (often called a *family*) of jobs (which means that they cannot process the other sets of jobs), and they do not always have the same processing times for the same job (see [Leung and Li \(2008\)](#) for a survey on scheduling problems with processing set restrictions).

Moreover, the photolithography area is usually a critical zone because it has the most expensive machines, which are consequently in limited number. To avoid idle machines, the area usually has a large number of lots waiting to be processed. Hence, any improvement in the performance of this area has a significant impact on the whole plant.

The photolithography scheduling problem is a parallel machine scheduling problem subject to auxiliary resource constraints. [Cakici and Mason \(2007\)](#) show that minimizing the weighted sum of completion times is NP-hard. They note the problem $P_m|r_j, aux|\sum w_j C_j$ and propose a mixed integer linear programming (MIP) model. They consider identical machines and release dates, whereas our paper deals with unrelated parallel machines with eligibility constraints and no release dates but sequence-dependent setup times. Furthermore, two objective functions are studied in our work: maximizing the number of produced wafers and minimizing the total weighted completion time. [Cakici and Mason \(2007\)](#) only consider the total weighted completion time. To solve the problem, they use an exact approach, an MIP model with disjunctive variables, and heuristics to decrease the solution time and to compare with the MIP model. Problems with 3 machines, 15 jobs, and 3 auxiliary resources are solved. Problems related to send-ahead wafers while minimizing cycle times are discussed by [Akçali et al. \(2001\)](#). A genetic algorithm is proposed by [Mönch \(2002\)](#) to solve the photolithography scheduling problem that takes into account send-ahead wafers, which are sent to control the machine status. [Kempf et al. \(1998\)](#) discuss a scheduling problem on a single burn-in oven, where auxiliary resources are load boards. The problem is formulated as $1|p - batch, incompatible, s, aux|\sum C_i$. [Diaz et al. \(2005\)](#) use discrete event simulation to show the impact of reticles on key indicators such as work in process and customer due date satisfaction. They also describe a modified apparent tardiness cost (ATC) algorithm to solve their problem. [Chandra et al. \(2008\)](#) consider the problem of scheduling semiconductor burn-in operations, where the burn-in oven is modeled as a batch processing machine. In this study, both oven capacity and the number of available boards are taken as constraints. The goal is to minimize the completion time of all jobs, and variants of a tabu search method are used.

In this article, jobs are processed on unrelated parallel machines with auxiliary resources, and setup times are

both sequence dependent and machine dependent. Moreover, there are machine eligibility constraints, and two objective functions are considered. The main originality of the problem comes from eligibility constraints and auxiliary resources. In addition, one of the criteria is not classical in scheduling theory but relevant in practice. The aim of the paper is to propose a metaheuristic that provides good solutions on large instances, namely, a *memetic algorithm*. Interesting properties of this method are presented and analyzed in the article.

A memetic algorithm is an *evolutionary algorithm*, inspired by evolutionary theory, to handle solutions so as to modify them and converge toward optimal solutions. It handles *populations* of solutions, i.e., a set of solutions that will iteratively be *selected*, *crossed*, and *mutated* to improve them according to a given criterion. Different strategies exist for performing these operations as well as for choosing the population for the following iteration. The mutation operation in a memetic algorithm consists of a local search method in which, at each iteration, we move from one solution to another by a transformation, called a *neighborhood operation*; the obtained solution is called a *neighbor solution*. A *stopping criterion*, i.e., a condition that must be satisfied to end the algorithm, must be defined. Some properties linked to the structure and specificities of the problem are integrated in the memetic algorithm. The choices that were made concerning the coding structure of the solution, the local search neighborhood operators, and the algorithm itself are properly motivated. The use of a memetic algorithm appears to fit our problem because of the coding structure that naturally integrates the setup and auxiliary resource constraints. Furthermore, the neighborhoods of the local search ensure properties that make the method efficient.

The article is organized as follows. Section 2 gives notations and formalizes the problem. The properties of the memetic algorithm and its coding are given in Sects. 3 and 4. The properties of the local search method are detailed in Sect. 5, and important properties can be found in Sect. 6. Some numerical experiments are discussed in Sect. 7. Finally, we conclude and give perspectives in Sect. 8.

2 Problem definition

In what follows, we describe the considered scheduling problem. First, some notations are given, then the constraints and two objective functions.

2.1 Notations

Let us consider a set $\mathcal{J}$ of N jobs, a set $\mathcal{M}$ of M machines, and a set $\mathcal{A} = \{A_1, \dots, A_\ell\}$ ($\ell \leq N$) of ℓ auxiliary resources (also called masks) necessary to process jobs on machines. The notations below are used throughout the paper.

- For each job $J_i \in \mathcal{J}$, its number of wafers $c_i \in \mathbb{N}$, its priority w_i , the index of the mask $\varphi_i \in \{1, \dots, \ell\}$ required for this job, and its set of qualified-machine indices $\mathcal{M}_i \subset \{1, \dots, M\}$.
- For each job $J_i \in \mathcal{J}$ and each machine M_j , $j \in \mathcal{M}_i$, the processing time $\rho_{ij} \in \mathbb{N}$ of job J_i on machine M_j , which is also proportional to the number of wafers c_i of J_i .
- For each job $J_i \in \mathcal{J}$, each job $J_k \in \mathcal{J}$, and each machine M_j such that $j \in \mathcal{M}_i \cap \mathcal{M}_k$, the setup time $\beta_{ijk} \in \mathbb{N}$ required by M_j to process J_k immediately after J_i .
- For each mask $A_i \in A$, its initial location $R_i \in \{0, \dots, M\}$, the index of the machine where the mask is initially located; 0 stands for the stocking/storing place.
- A time horizon $H \in \mathbb{N}$.

2.2 Constraints

The constraints of the problem are as follows:

- A job is processed once and only once on a qualified machine for this job.
- The start times are integers, and all machines are available at time 0.
- There is no preemption, i.e., a job is not interrupted after it has started.
- A machine can only process one job at a time.
- Two jobs J_i and J_k having the same required mask (i.e., $\varphi_i = \varphi_k$) cannot be processed at the same time on two different machines. Indeed, masks are shared resources between jobs, and processing a job on a given machine implies a *move* of the required mask toward the machine, if the mask is not already loaded in the machine.
- A mask move from one machine to another (even the storage place) implies a unitary transport time.

2.3 Criteria

The specificities of this problem and its implementation in an industrial context led us to study two different criteria: maximization of the number of produced wafers and minimization of the weighted completion times. The second one is well known in the scheduling literature.

- The number of produced wafers is an important criterion, and, since there is a continuous flow of arriving jobs in the workshop, a time horizon is given to fix a short-term productivity goal to reach. Indeed, we need to take into account new jobs that arrive after some period of time. Thus, this first criterion is only maximized on a given time horizon H . If in a solution we denote by m_i the index of the assigned machine to job J_i and its start time as t_i , then we can express the number of produced wafers before H as follows:

$$\sum_{i=1}^N c_i \theta_i,$$

where θ_i is the *ratio* of job J_i that is processed before H , over its processing time:

$$\theta_i = \begin{cases} \frac{\min(\rho_{im_i}, H-t_i)}{\rho_{im_i}} & \text{if } t_i \leq H, \\ 0 & \text{if } t_i > H. \end{cases}$$

Using this formula, we express the fact that the number of produced wafers during the processing of a job is proportional to the elapsed time. If a job is only half-processed before H , only half of its number of wafers is counted in the cost of the solution. For example, with 25 wafers in a job, we add 12.5 to the cost. Hence, the number of wafers processed before H is calculated assuming that wafers of the same job have the same processing time.

- Since there are job priorities associated to the job waiting times in the workshop and customer priorities, the weighted sum of completion times is also considered. Using the same notations, we denote by C_i the completion time of job J_i , i.e., $C_i = t_i + \rho_{im_i}$, and the objective function to minimize is

$$\sum_{i=1}^N w_i C_i.$$

For both problems (criteria), every solution is written $((m_1, t_1), \dots, (m_N, t_N)) \in \chi$, where, for all $i \in \{1, \dots, N\}$, $m_i \in \mathcal{M}_i$ is the index of the machine that processes J_i and $t_i \in \mathbb{N}$ is its start time.

Let us denote by Π^1 the problem where the number of wafers produced before H is maximized and by Π^2 the problem where the weighted sum of completion times is minimized.

3 Coding structure and properties

In a memetic algorithm, operations are not directly performed on the elements of the solution space. These elements are usually represented in a set, the set of individuals, and a *decoding* algorithm is used to build a solution from each individual. In what follows, $\mathcal{C}$ denotes the set of individuals used in the algorithm.

The coding of one solution is as follows: $(x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$, where x_i is a job index $[(x_1, \dots, x_N)]$ is a permutation of the set $\{1, \dots, N\}$ and y_i is a machine index (qualified for job J_{x_i}).

Then, for each element $(x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$, $\{x_1, \dots, x_N\} = \{1, \dots, N\}$ and $y_i \in \mathcal{M}_{x_i}$.

For each individual $(x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$, there exists one and only one corresponding solution $z \in \chi$. Let us define the mapping

$$f : \mathcal{C} \rightarrow \chi$$

$$(x_1, \dots, x_N, y_1, \dots, y_N) \mapsto f(x_1, \dots, x_N, y_1, \dots, y_N),$$

which is here described by Algorithm 1. For all $X \in \mathcal{C}$, this algorithm returns $f(X) \in \chi$. Recall that every solution of Π^1 (resp. Π^2) is written

$$((m_1, t_1), \dots, (m_N, t_N)) \in \chi,$$

where, for all $i \in \{1, \dots, N\}$, $m_i \in \mathcal{M}_i$ is the index of the machine that processes J_i , and $t_i \in \mathbb{N}$ is its start time. In what follows, I denotes an instance of the problem as a parameter of the algorithms.

```

1 DECODE( $I, x_1, \dots, x_N, y_1, \dots, y_N$ )
2  $t_0 \leftarrow 0$ 
3 for  $j \leftarrow 0$  to  $M$  do
4    $\rho_{0j} \leftarrow 0$ 
5 for  $j \leftarrow 1$  to  $N$  do
6    $\rho_{j0} \leftarrow 0$ 
7 for  $i \leftarrow 1$  to  $\ell$  do
8    $S[i] \leftarrow R_i$   $\triangleright$  Current position of mask
9    $B[i] \leftarrow 0$   $\triangleright$  Last job processed with mask
10 for  $i \leftarrow 1$  to  $M$  do
11    $D[i] \leftarrow 0$   $\triangleright$  Last job scheduled on
      machine
12 for  $i \leftarrow 1$  to  $N$  do
13    $\tau \leftarrow 0$ 
14   if  $S[\varphi_{x_i}] \neq y_i$  then
15      $\tau \leftarrow 1$   $\triangleright$  Unitary time to move a mask
16    $t_{x_i} \leftarrow \max(t_{B[\varphi_{x_i}]} + \rho_{B[\varphi_{x_i}]} S[\varphi_{x_i}] + \tau, t_{D[y_i]} + \rho_{D[y_i]} y_i + \beta_{D[y_i]} y_i x_i)$ 
17    $D[y_i] \leftarrow x_i$   $\triangleright$  Update last job processed
      on machine
18    $B[\varphi_{x_i}] \leftarrow x_i$   $\triangleright$  Update last job processed
      with mask
19    $S[\varphi_{x_i}] \leftarrow y_i$   $\triangleright$  Update position of mask
20 for  $i \leftarrow 1$  to  $N$  do
21    $m_{x_i} \leftarrow y_i$ 
22 return  $((m_1, t_1), \dots, (m_N, t_N))$ 

```

Algorithm 1: Decoding algorithm

Note that in $(x_1, \dots, x_N, y_1, \dots, y_N)$, y_i is the index of the machine that is assigned to job J_{x_i} in the solution $f(x_1, \dots, x_N, y_1, \dots, y_N)$.

Definition 1 A solution $x = ((m_1, t_1), \dots, (m_N, t_N)) \in \chi$ is an *early schedule* if and only if for all $i \in \{1, \dots, N\}$ we cannot decrease t_i without modifying the start times of other jobs, the sequence of jobs on machine M_{m_i} , the sequence of jobs that require mask A_{φ_i} , the assignment of these jobs on their machine, and the feasibility of the solution, i.e., there is no solution $((m_1, t_1), \dots, (m_i, t'_i), \dots, (m_N, t_N)) \in \chi$ such that the jobs scheduled on M_{m_i} are in the same order as in x ,

jobs J_k , such that $\varphi_k = \varphi_i$, are processed in the same order as in x and $t'_i < t_i$.

Proposition 1 For each element $(x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$,

$$((m_1, t_1), \dots, (m_N, t_N)) = f(x_1, \dots, x_N, y_1, \dots, y_N) \in \chi,$$

and it is an early schedule.

Proof Let $X = (x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$. We first show that Algorithm DECODE (Algorithm 1) returns a solution in χ . According to Line 21 and the definition of $\mathcal{C}$, the machine qualification constraint is satisfied. Each job is processed once and only once.

The setup time constraints, the nonsimultaneous processing of jobs on a single machine, and mask constraints are always satisfied because of Line 16. It expresses that, to start a job on a given machine, two conditions are necessary (expressed by max): the availability of the required mask (i.e., the completion time of the last job processed with this mask plus 1 if a mask move is necessary) and the availability of the machine (i.e., the completion time of the last job processed on the machine plus a setup time).

Then $f(X) \in \chi$. Let us show that this is an early schedule.

Let $i \in \{1, \dots, N\}$; if we replace the start time of job J_i with $t_i - K$ (K strictly positive) and retain the same values for the m_j and t_j ($j \neq i$), then at least one of the two previous conditions is no longer satisfied (the required mask is not available yet, the machine is busy, or maybe both). Indeed, job J_i must be processed after the last job J_j processed on machine m_i and after the last job J_k such that $\varphi_k = \varphi_i$. The algorithm allows J_i to be processed at the earliest time after both J_j and J_k are completed.

We conclude that $f(X)$ is an early schedule. $\square$

Proposition 2 Early schedules are dominant solutions for Π^1 and Π^2 .

Indeed, the two criteria are regular, so decreasing the start time of one job cannot decrease the quality of a solution.

Theorem 1 In $f(\mathcal{C})$, there exists an optimal solution for Π^1 (resp. Π^2).

Proof Let $\chi_t \subset \chi$ be the set of early schedules. We want to show that $f(\mathcal{C}) = \chi_t$. The inclusion $f(\mathcal{C}) \subset \chi_t$ is given by Proposition 1. Let us show that $\chi_t \subset f(\mathcal{C})$.

Let $Y = ((m_1, t_1), \dots, (m_N, t_N)) \in \chi_t$ denote an early schedule, and let $\sigma_1, \dots, \sigma_N$ denote indices $\{1, \dots, N\}$ such that $t_{\sigma_1} \leq \dots \leq t_{\sigma_N}$. Then the individual $X = (\sigma_1, \dots, \sigma_N, m_{\sigma_1}, \dots, m_{\sigma_N}) \in \mathcal{C}$ is such that $f(X) = Y$. Indeed, in Algorithm 1, jobs are scheduled in the order given by $\sigma_1, \dots, \sigma_N$, and the start times are the earliest ones. Then, in $f(X)$, for jobs processed on the same machine, the sequence is the same as in Y on each machine. And

since the start times are the earliest possible times, they are the same. Furthermore, jobs having the same required mask are processed in the same order, given that they cannot be processed in parallel. Their start times are then the same. If t_i and t_j are equal, then jobs J_i and J_j are not processed on the same machine and do not have the same required mask, according to the constraints, so the obtained solution will be the same regardless of the chosen order between i and j in the coding. $\square$

This result allows us to justify the coding we used, as well as the decoding algorithm. Now that we have an appropriate coding structure, we propose a memetic algorithm and use the coding structure to return a feasible solution for Π^1 and Π^2 .

4 Memetic algorithm

Algorithm 2 describes the steps of the solving method. The inputs are as follows: an instance I of Π^1 (resp. Π^2), the size N_{pop} of the population, the number n_R of iterations without improvement before resetting, the number n_A of iterations without improvement before stopping, the integer K which will be used at the selection step, the objective function ($f_{obj} = 1$ for Π^1 , $f_{obj} = 2$ for Π^2), the rate t_D of individuals (out of N) created by crossover (offspring), the rate t_S of individuals (over the created individuals) that remain in the offspring at the next iteration, the number ItR of iterations in the local search, the integers (positive) p_1 and p_2 ($p_1 + p_2 \leq 100$), the real $T > 0$, the integer n_p , and the real $\alpha \in [0, 1]$ used in Algorithm LOCAL-SEARCH (introduced in Sect. 5).

As shown in Algorithm 2, the size of the population remains constant. A given percentage of the population is created by crossover (t_D), and only the best individuals are kept (in proportion t_S of the number of generated individuals). A counter is used for the number of nonimproving iterations; it defines the stopping criterion of the algorithm. When n_A consecutive iterations do not improve the best known solution, the algorithm stops. Another counter is used to reset the population after n_R consecutive iterations without any improvement in the best solution found since the last reset. This is why two solutions are retained: the best known solution since the beginning and the best one since the last reset. c_{best}^{Rein} is the cost of solution $bestRein$, i.e., the best solution since the last population reset, whereas c_{best} is the cost of solution $best$, i.e., the best solution found so far by the algorithm.

Sorting population The SORT procedure simply sorts the input according to the evaluation (or fitness) of the solution, with the considered objective function. If $f_{obj} = 1$, then

```

1 MEMETIC( $I, N_{pop}, n_R, n_A, K, f_{obj}, t_D, t_S, ItR, p_1, p_2, T, n_p, \alpha$ )
2 for  $i \leftarrow 1$  to  $N_{pop}$  do
3    $P[i] \leftarrow \text{INITIALIZE}(I)$ 
4    $P[i] \leftarrow \text{LOCAL-SEARCH}(I, P[i], ItR, f_{obj}, p_1, p_2, T, n_p, \alpha)$ 
5  $nbItR \leftarrow 0$ 
6  $nbItA \leftarrow 0$ 
7  $\text{SORT}(P, f_{obj})$ 
8  $best \leftarrow P[1]$   $\triangleright$  Best known solution
9  $c_{best} \leftarrow \text{EVALUATE}(I, best, f_{obj})$ 
10  $bestRein \leftarrow P[1]$   $\triangleright$  Best known solution since
    reset
11  $c_{best}^{Rein} \leftarrow c_{best}$ 
12 while  $nbItA < n_A$  do
13   for  $i \leftarrow 1$  to  $\lfloor N \times t_D \rfloor$  do
14      $p_1 \leftarrow \text{SELECTION}(P, K)$ 
15      $p_2 \leftarrow \text{SELECTION}(P, K)$ 
16      $Desc[i] \leftarrow \text{CROSSOVER}(p_1, p_2)$ 
17      $Desc[i] \leftarrow \text{LOCAL-SEARCH}(I, Desc[i], ItR, f_{obj}, p_1, p_2, T, n_p, \alpha)$ 
18    $\text{SORT}(Desc, f_{obj})$   $\triangleright$  Sorting before
    insertion of offspring
19   for  $i \leftarrow 1$  to  $\lfloor \lfloor N \times t_D \rfloor \times t_S \rfloor$  do
20      $P[N_{pop} - i + 1] \leftarrow Desc[i]$ 
21    $\text{SORT}(P, f_{obj})$   $\triangleright$  Sorting before handling
    best solution
22    $evalact \leftarrow \text{EVALUATE}(I, P[1], f_{obj})$ 
23   if ( $f_{obj} = 1$  and  $c_{best} < evalact$ ) or ( $f_{obj} = 2$  and
     $c_{best} > evalact$ ) then
24      $best \leftarrow P[1]$   $\triangleright$  Updating best solution
25      $c_{best} \leftarrow evalact$ 
26      $nbItA \leftarrow 0$ 
27   else
28      $nbItA \leftarrow nbItA + 1$ 
29   if ( $f_{obj} = 1$  and  $c_{best}^{Rein} < evalact$ ) or ( $f_{obj} = 2$  and
     $c_{best}^{Rein} > evalact$ ) then
30      $bestRein \leftarrow P[1]$   $\triangleright$  Updating best
    solution
31      $c_{best}^{Rein} \leftarrow evalact$ 
32      $nbItR \leftarrow 0$ 
33   else
34      $nbItR \leftarrow nbItR + 1$ 
35   if  $nbItR = n_R$  then
36      $\text{RESET}(P)$   $\triangleright$  Resetting population
37     for  $i \leftarrow 1$  to  $N$  do
38        $P[i] \leftarrow \text{LOCAL-SEARCH}(I, P[i], ItR, f_{obj}, p_1, p_2, T, n_p, \alpha)$ 
39      $nbItR \leftarrow 0$ 
40      $\text{SORT}(P, f_{obj})$ 
41      $bestRein \leftarrow P[1]$ 
42      $c_{best}^{Rein} \leftarrow \text{EVALUATE}(I, bestRein, f_{obj})$ 
43 return  $\text{DECODE}(I, best)$ 

```

Algorithm 2: Memetic algorithm

individuals are sorted in decreasing order of fitness, otherwise in increasing order, so that the best solution is always at index 1.

Fitness The fitness of an individual X is simply the cost of the corresponding solution $f(X)$ obtained by the decoding algorithm. The function $\text{EVALUATE}(I, z, f)$ returns the cost

of the solution obtained from individual z , according to the objective function f ($f = 1$ for the number of produced wafers and $f = 2$ for the weighted sum of completion times), for instance I . This function calculates the fitness of each individual.

Initializing Initializing (as resetting) is based on a randomized algorithm to generate an element of the set $\mathcal{C}$. The job sequence is randomly generated by iteratively selecting an index in $\{1, \dots, N\}$ (that has not been selected yet), based on a uniform law, until each index is selected. Then, for each job, a qualified machine index is randomly picked.

The reset procedure is similar. Algorithm INITIALIZE is used N times.

Selection Selection is carried out by K -tournament: K individuals of the population are randomly selected based on a uniform law, and the best one is the selected individual.

Crossover The crossover procedure with individuals u and u' consists of randomly choosing a vector $v = (v_1, \dots, v_N) \in \{0, 1\}^N$ to determine (using the indices i such that $v_i = 1$) the n_1 indices of $(x_1, \dots, x_N)$ that generate (in the same order as in u) the n_1 first values in the son. The other indices (in $\{1, \dots, N\}$) are in the same order as in u' for the $n - n_1$ last indices. This crossover procedure is well known and very classical in the metaheuristic literature.

Example 1 Let us consider the following example, with $N = 5$, $v = \{1, 0, 1, 0, 1\}$, and the following individuals:

$$u = (x_1, \dots, x_5, y_1, \dots, y_5) = (4, 3, 2, 5, 1, 1, 1, 2, 1, 2)$$

and

$$u' = (x'_1, \dots, x'_5, y'_1, \dots, y'_5) = (5, 4, 1, 3, 2, 1, 2, 2, 1, 2),$$

then $u'' = (4, 2, 1, 5, 3, 1, 2, 2, 1, 1)$.

The general sketch of the algorithm is now set. The details of the local search procedure are now provided. This final step provides the entire description of the algorithm.

5 Local search

Algorithm 3 shows how a mutation is performed on a generated son, by iteratively improving the corresponding solution. Here, we consider that $\text{RAND}()$ [resp. $\text{RANDOM}(a, b)$] is a real (resp. an integer) number between 0 and 1 (resp. between a and b), randomly selected based on a uniform law.

During ItR iterations, an individual $u \in \mathcal{C}$ is modified by three operations: V_1 , V_2 , and V_3 . At each iteration, one of these operations is randomly chosen: V_1 with probability $\frac{p_1}{100}$,

```

1 LOCAL-SEARCH( $I, u, ItR, f_{obj}, p_1, p_2, T, n_p, \alpha$ )
2  $c \leftarrow \text{EVALUATE}(I, u, f_{obj})$ 
3  $best \leftarrow u$ 
4  $c_{best} \leftarrow c$ 
5 for  $i \leftarrow 1$  to  $ItR$  do
6    $p \leftarrow \text{RANDOM}(1, 100)$ 
7   if  $p \leq p_1$  then
8      $u' \leftarrow V_1(I, u)$ 
9   else
10    if  $p \leq p_1 + p_2$  then
11       $u' \leftarrow V_2(I, u)$ 
12    else
13       $u' \leftarrow V_3(I, u)$ 
14     $c' \leftarrow \text{EVALUATE}(I, u', f_{obj})$ 
15    if ( $f_{obj} = 1$  and  $c < c'$ ) or ( $f_{obj} = 2$  and  $c > c'$ ) then
16       $c \leftarrow c'$ 
17       $u \leftarrow u'$ 
18    else
19       $\Delta \leftarrow |c - c'|$ 
20       $p \leftarrow \text{RAND}()$ 
21      if  $p < \exp^{-\frac{\Delta}{T}}$  then
22         $c \leftarrow c'$ 
23         $u \leftarrow u'$ 
24    if  $i \equiv 0 \pmod{n_p}$  then
25       $T \leftarrow \alpha T$   $\triangleright$  Decreasing  $T$  every  $n_p$ 
        iterations
26    if ( $f_{obj} = 1$  and  $c_{best} < c$ ) or ( $f_{obj} = 2$  and  $c_{best} > c$ ) then
27       $c_{best} \leftarrow c$ 
28       $best \leftarrow u$ 
29 return  $best$ 

```

Algorithm 3: Local search

V_2 with probability $\frac{p_2}{100}$, and V_3 with probability $1 - \frac{p_1 + p_2}{100}$. The resulting individual $u' \in \mathcal{C}$ is maintained if it has a better fitness (Lines 15–17). Otherwise, i.e., if the fitness is not improved, it will be maintained with probability $\exp(-\frac{\Delta}{T})$, where T is a parameter and Δ is the difference between the fitness of u and u' . The value of T is gradually decreased (every n_p iterations, by multiplying by α , $0 \leq \alpha \leq 1$), so the probability of maintaining a solution u' with a worse fitness decreases. The returned individual is the best ever built.

In what follows, the neighborhood operations V_1 , V_2 , and V_3 are described:

$$\begin{aligned}
 V_1 : \mathcal{C} \times \{1, \dots, N\} \times \{1, \dots, M\} &\rightarrow \mathcal{C} \\
 ((x_1, \dots, x_i, \dots, x_N, y_1, \dots, y_i, \dots, y_N), i, y) & \\
 \mapsto (x_1, \dots, x_i, \dots, x_N, y_1, \dots, y, \dots, y_N). &
 \end{aligned}$$

V_1 consists in replacing one component (among the N last ones) with a value $y \in \{1, \dots, M\}$. It is the *reassignment* of one job to another qualified machine.

Example 2 With $N = 4$, $M = 3$,

$$V_1((1, 4, 3, 2, 1, 1, 1, 1), 2, 3) = (1, 4, 3, 2, 1, 3, 1, 1).$$

$$\begin{aligned}
V_2 : \mathcal{C} \times \{1, \dots, N\} \times \{1, \dots, N+1\} &\rightarrow \mathcal{C} \\
((x_1, \dots, x_i, \dots, x_j, \dots, x_N, y_1, & \\
\dots, y_i, \dots, y_j, \dots, y_N), i, j) & \\
\mapsto & \\
(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_{j-1}, x_i, x_j, & \\
\dots, x_N, y_1, \dots, y_{i-1}, y_{i+1}, \dots, y_{j-1}, y_i, y_j, & \dots, y_N).
\end{aligned}$$

$V_2(u, i, j)$ consists in *moving* the i th (resp. the $N + i$ th) component of u at the index before the j th (resp. the $N + j$ th) component in u . When $j = 1$, the new index of the component is 1 (resp. $N + 1$). When $j = N + 1$, it is N (resp. $2N$).

Example 3 With $N = 4, M = 3$,

$$V_2((1, \mathbf{4}, 3, 2, 1, \mathbf{2}, 1, 1), 2, 5) = (1, 3, 2, \mathbf{4}, 1, 1, \mathbf{1}, \mathbf{2}),$$

also

$$V_2((1, \mathbf{4}, 3, 2, 1, \mathbf{2}, 1, 1), 2, 4) = (1, 3, \mathbf{4}, 2, 1, 1, \mathbf{2}, 1)$$

and

$$V_2((1, \mathbf{4}, 3, 2, 1, \mathbf{2}, 1, 1), 2, 1) = (\mathbf{4}, 1, 3, 2, \mathbf{2}, 1, 1, 1).$$

$$\begin{aligned}
V_3 : \mathcal{C} \times \{1, \dots, N\} \times \{1, \dots, N\} &\rightarrow \mathcal{C} \\
((x_1, \dots, x_i, \dots, x_j, \dots, x_N, y_1, \dots, y_i, \dots, & \\
y_j, \dots, y_N), i, j) & \\
\mapsto & \\
(x_1, \dots, x_j, \dots, x_i, \dots, x_N, y_1, \dots, y_j, \dots, y_i, \dots, y_N).
\end{aligned}$$

V_3 consists in *swapping* two components (the i th and j th, the $N + i$ th, and $N + j$ th).

Example 4 With $N = 4, M = 3$,

$$V_3((1, \mathbf{4}, 3, \mathbf{2}, 1, \mathbf{2}, 1, 1), 2, 4) = (1, \mathbf{2}, 3, \mathbf{4}, 1, \mathbf{1}, \mathbf{1}, \mathbf{2}),$$

In what follows, the chosen neighborhoods and the scheme of the local search algorithm are motivated.

6 Properties

To introduce interesting properties of the neighborhoods, here is a preliminary definition.

Definition 2 Let $u = (x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$. Two components, x_i and x_j , of u are *related according to u* if one of the following conditions is true:

- Jobs J_{x_i} and J_{x_j} require the same mask: $\varphi_{x_i} = \varphi_{x_j}$.
- Jobs J_{x_i} and J_{x_j} are processed on the same machine in solution $f(u)$: $y_i = y_j$.

This definition applies to a set of components $\{x_{i_1}, \dots, x_{i_s}\}$ that are related according to u if one of these conditions is true:

- Jobs $J_{x_{i_1}}, \dots, J_{x_{i_s}}$ require the same mask: $\varphi_{x_{i_1}} = \dots = \varphi_{x_{i_s}}$.
- Jobs $J_{x_{i_1}}, \dots, J_{x_{i_s}}$ are processed on the same machine in solution $f(u)$: $y_{i_1} = \dots = y_{i_s}$.

Proposition 3 For all $u \in \mathcal{C}$, $i, j \in \{1, \dots, N\}$, $i \leq j$,

$$V_3(u, i, j) = V_2(V_2(u, i, j), j, i).$$

Proof Component b (resp. a) with index j (resp. i) in u will have index j (resp. $j - 1$) in $u' = V_2(u, i, j)$. Since $i \leq j$, in $V_2(u', j, i)$, a will be at index j and b will be at index i . The other components keep the same position as in u [those at indices lower than i or greater than j never move; those at indices between i and j decrease their index by one in u' and increase by one in $V_2(u', j, i)$]. $\square$

In what follows, we justify the neighborhoods used in the solving method.

Proposition 4 For each couple $(u_1, u_2) \in \mathcal{C}^2$ of individuals, there exist integers $i_1, \dots, i_s, l_1, \dots, l_s, j_1, \dots, j_r, k_1, \dots, k_r$ such that

$$\begin{aligned}
u_2 = V_2(\dots, (V_2(V_1(\dots (V_1(u_2, j_1, k_1), \dots) & \\
, j_r, k_r), i_1, l_1), \dots), i_s, l_s).
\end{aligned}$$

In other words, there exists a sequence of neighborhood operations from u_1 to u_2 .

Proof Let $u_1 = (x_1, \dots, x_N, y_1, \dots, y_N)$ and $u_2 = (x'_1, \dots, x'_N, y'_1, \dots, y'_N)$ be two elements of $\mathcal{C}$. According to the definition of $\mathcal{C}$, $\{x_1, \dots, x_N\} = \{x'_1, \dots, x'_N\} = \{1, \dots, N\}$. For all $i \in \{1, \dots, N\}$, let σ_i denote the index such that $x'_{\sigma_i} = x_i$.

Then we consider

$$\begin{aligned}
u_3 = V_1(\dots (V_1(u_1, 1, y'_{\sigma_1}), \dots), n, y'_{\sigma_N}) & \\
= (x_1, \dots, x_N, y'_{\sigma_1}, \dots, y'_{\sigma_N}).
\end{aligned}$$

An important result in group theory is that every element of the symmetric group of the set $\{1, \dots, N\}$, i.e., every permutation of $\{1, \dots, N\}$, can be written as a finite product of transpositions. This implies that there exist indices $i_1, \dots, i_s, j_1, \dots, j_s$ such that

$$\begin{aligned}
u_2 = (x'_1, \dots, x'_N, y'_1, \dots, y'_N) & \\
= V_3(\dots (V_3(u_3, i_1, j_1), \dots), i_s, j_s),
\end{aligned}$$

which implies, according to Proposition 3, that there exist indices $i'_1, \dots, i'_{2s}, j'_1, \dots, j'_{2s}$ such that

$$\begin{aligned} u_2 &= (x'_1, \dots, x'_N, y'_1, \dots, y'_N) \\ &= V_2(\dots (V_2(u_3, i'_1, j'_1), \dots), i'_{2s}, j'_{2s}), \end{aligned}$$

which completes the proof. $\square$

Theorem 2 *There exists an optimal solution in χ that can be returned by the MEMETIC algorithm for problem Π^1 (resp. Π^2).*

Proof Immediate, according to the following three assertions:

- Theorem 1 states that there exists $y \in \mathcal{C}$ such that $f(y)$ is optimal for Π^1 (resp. Π^2).
- Algorithm MEMETIC (with LOCAL- SEARCH procedure) is designed in such a way that there is a nonnull probability of switching from one individual to another via a neighborhood operation.
- Proposition 4 shows that each $u \in \mathcal{C}$ can be computed by the local search method.

Then the *connectivity* of the neighborhoods is satisfied. We note that f (which is the function that returns, for each individual, an early schedule) is not injective. In other words, two or more individuals can have the same image by f .

Definition 3 Let $u \in \mathcal{C}$. Let us call a *meaningful neighbor* of u each $u' \in \mathcal{C}$ such that u' is a neighbor of u and $f(u) \neq f(u')$.

In the memetic algorithm, the following property can be used.

Proposition 5 *Let $u = (x_1, \dots, x_N, y_1, \dots, y_N) \in \mathcal{C}$ and $i \in \{1, \dots, N\}$. Let i_1 and i_2 be two elements in $\{1, \dots, N\}$ such that $i_1 \leq i \leq i_2$ and for all $k \in \{i_1, \dots, i_2\} \setminus \{i\}$, $y_k \neq y_i$ and $\varphi_{x_k} \neq \varphi_{x_i}$.*

Then the following equality holds: for all $k \in \{i_1, \dots, i_2\}$,

$$f(V_2(u, i, k)) = f(u).$$

Proof According to the definition of i_1 and i_2 , and using the DECODE algorithm, the order of the related indices according to u is the same in u and $V_2(u, i, k)$.

The start times are the earliest possible times in the DECODE algorithm such that the sequence of jobs on the same machine and the sequence of jobs that require the same mask do not change. Hence, the DECODE algorithm returns the same solutions:

$$f(V_2(u, i, k)) = f(u).$$

Then it is possible to slightly modify the second neighborhood operator in order to add this test. Indeed, instead of

randomly searching the index to move, we can compute, for a given $u \in \mathcal{C}$ and $i \in \{1, \dots, N\}$, the indices i_1 and i_2 such that $i_1 \leq i \leq i_2$ for all $k \in \{i_1, \dots, i_2\} \setminus \{i\}$, $y_k \neq y_i$ and $\varphi_{x_k} \neq \varphi_{x_i}$, and if $i_1 > 1$, then $y_{i_1-1} = y_i$ or $\varphi_{x_{i_1-1}} = \varphi_{x_i}$, and if $i_2 < n$, then $y_{i_2+1} = y_i$ or $\varphi_{x_{i_2+1}} = \varphi_{x_i}$. Then a search for the index to move can be carried out by randomly choosing from the set $\{1, \dots, i_1 - 1\} \cup \{i_2 + 1, \dots, N\}$.

7 Numerical results

Numerical experiments to analyze the performance of our memetic algorithm were performed on an Intel Core i7-2640M CPU, 2.80 GHz, with a C++ implementation, on generated instances of the problem. These instances are described as $N - M - \ell - H$, where N is the number of jobs, M the number of machines, ℓ the number of masks, and H the time horizon. There are ten instances per type, which are generated as follows (random generations are following a uniform law):

- Processing times are randomly generated in the interval $[1, 10]$.
- Setup times are randomly generated in the interval $[1, 5]$.
- For each job, its number of qualified machines is randomly chosen (between 1 and M), and the set of qualified machines is randomly selected.
- For the first ℓ jobs, the required mask of the i th job is the i th mask. For the remaining jobs, the required mask index is randomly selected.
- The number of wafers of each job is 25 with a probability of 90% and a random number between 1 and 25 with a probability of 10%.
- The priorities are randomly selected in the interval $[1, 10]$.
- The initial location of each mask is randomly selected in the interval $[0, M]$.

We want to analyze the impact on the solution quality of the population size and of how the local search neighborhoods are activated. The other parameters of the algorithm are fixed to classical values in genetic algorithms as follows:

- $t_D = 50\%$: only half of the individuals are chosen for crossover;
- $t_S = 80\%$: among the remaining sons, the best 80 % are retained to avoid changing the next generation too much, which means 40 % of the population is changed;
- $K = \frac{N_{pop}}{4}$: the number of individuals from among which the selected ones are chosen must not be too large; otherwise, some individuals will never be selected;

Table 1 Quality (mean of ten instances) of solutions for Π^1 ($\max \sum_{i=1}^N c_i \theta_i$) based on population size ($p_1 = 45\%$, $p_2 = 45\%$)

$(N - M - \ell - H)$	N_{pop}		
	30	60	100
10 – 2 – 3 – 25	206 (5s)	206 10s	206 (16s)
20 – 2 – 3 – 50	439 (14s)	441 (26s)	442 (36s)
30 – 3 – 4 – 50	643 (30s)	647 (65s)	654 (102s)
40 – 3 – 4 – 60	791 (64s)	789 (94s)	793 (188s)
50 – 3 – 6 – 80	1052 (53s)	1071 (190s)	1078 (450s)
70 – 4 – 10 – 85	1551 (90s)	1572 (236s)	1586 (346s)
90 – 4 – 10 – 110	1967 (209s)	1982 (319s)	2037 (437s)
100 – 5 – 12 – 100	2166 (301s)	2218 (397s)	2257 (522s)
120 – 5 – 12 – 120	2512 (340s)	2650 (415s)	2707 (663s)
150 – 6 – 14 – 125	3215 (244s)	3406 (521s)	3397 (943s)
200 – 8 – 20 – 100	3833 (323s)	3962 (674s)	4000 (783s)

- $\alpha = 0.9$, $T = 100$, $ItR = 100$, $n_p = 5$: parameters of the simulated annealing procedure;
- $n_A = \min(\max(500, 3N), 800)$: number of iterations without improvements before stopping the algorithm; it must increase with n , but the bounds 500 and 800 were set to avoid this parameter to be too low or too large;
- $n_R = \frac{n_A}{5}$: number of iterations without improvements before resetting the population; it must be lower than n_A to guarantee at least some resets.

The results in Tables 1 and 2 show that, for Π^1 and Π^2 , most of the best solutions are obtained with $N_{pop} = 100$. Indeed, with 30 individuals, the diversification of the population is limited. It is better to use at least twice as many individuals. But it would be more time consuming to increase the size of the population by too much. For instance, it is not really necessary to exceed 60 for Π^1 since, in some instances, the solutions are even better than with $N = 100$ and, in the other instances, the gap between the solutions obtained with $N_{pop} = 60$ and with $N_{pop} = 100$ is not very large.

Tables 3 and 4 show the impact of neighborhood probabilities. For Π^1 , the best solutions for more than 90 % of the instances are obtained with $p_1 = 80\%$ and $p_2 = 10\%$. When the number of processed wafers before a time horizon H is maximized, it is very important to allow jobs to be assigned to another machine, and the corresponding probability should be large enough. The results are different for Π^2 : (10 %, 80 %) and (45 %, 45 %) are more effective since most of the instances are solved to their best known solution when $p_2 \geq 45\%$. The resequencing neighborhoods are more effective for Π^2 , i.e., when the sum of the weighted completion times is minimized. These results illustrate the fact that the best setting for these parameters differs according to the objective function.

Table 2 Quality (mean of ten instances) of solutions for Π^2 ($\min \sum_{i=1}^N w_i C_i$) based on population size ($p_1 = 45\%$, $p_2 = 45\%$)

$(N - M - \ell - H)$	N_{pop}		
	30	60	100
10 – 2 – 3 – 25	156 (5s)	156 (10s)	156 (16s)
20 – 2 – 3 – 50	529 (11s)	529 (21s)	526 (29s)
30 – 3 – 4 – 50	747 (36s)	746 (60s)	750 (101s)
40 – 3 – 4 – 60	1391 (47s)	1386 (94s)	1385 (130s)
50 – 3 – 6 – 80	1926 (59s)	1841 (148s)	1840 (389s)
70 – 4 – 10 – 85	2488 (105s)	2523 (212s)	2462 (490s)
90 – 4 – 10 – 110	3953 (187s)	3817 (293s)	3743 (532s)
100 – 5 – 12 – 100	4276 (256s)	4172 (513s)	4134 (845s)
120 – 5 – 12 – 120	6196 (220s)	5922 (418s)	5861 (417s)
150 – 6 – 14 – 125	7266 (360s)	7370 (566s)	7125 (850s)
200 – 8 – 20 – 100	8980 (402s)	8960 (525s)	8755 (855s)

Table 3 Quality (mean of ten instances) of solutions for Π^1 ($\max \sum_{i=1}^N c_i \theta_i$) based on neighborhood ($N_{pop} = 100$)

$(N - M - \ell - H)$	(p_1, p_2)		
	(10%, 80%)	(45%, 45%)	(80%, 10%)
10 – 2 – 3 – 25	206	206	206
20 – 2 – 3 – 50	425	441	453
30 – 3 – 4 – 50	636	654	659
40 – 3 – 4 – 60	795	793	809
50 – 3 – 6 – 80	1080	1078	1085
70 – 4 – 10 – 85	1581	1586	1581
90 – 4 – 10 – 110	2031	2037	2095
100 – 5 – 12 – 100	2256	2257	2284
120 – 5 – 12 – 120	2701	2707	2711
150 – 6 – 14 – 125	3337	3397	3412
200 – 8 – 20 – 100	3989	4000	4019

Finally, to analyze the impact of the test added based on Proposition 5, Table 5 shows the mean percentage of forbidden indices for a given job when a move neighborhood is called. This value is the mean of all the $\frac{i_2 - i_1 + 1}{N}$ computed in one call of the memetic algorithm. This percentage decreases when the instance gets larger, which is expected since there are more machines and masks, and thus fewer chances to see many jobs share the same mask or the same assigned machine. With 200 jobs, the mean is 7 %, which corresponds to approximately 14 forbidden indices each time the rule is applied. This is interesting because many useless neighborhoods are avoided.

Table 4 Quality (mean of ten instances) of solutions for Π^2 ($\min \sum_{i=1}^N w_i C_i$) based on neighborhood ($N_{pop} = 100$)

$(N - M - \ell - H)$	(p_1, p_2)		
	(10%, 80%)	(45%, 45%)	(80%, 10%)
10 – 2 – 3 – 25	156	156	156
20 – 2 – 3 – 50	541	526	563
30 – 3 – 4 – 50	778	750	788
40 – 3 – 4 – 60	1377	1385	1395
50 – 3 – 6 – 80	1853	1840	1851
70 – 4 – 10 – 85	2445	2462	2462
90 – 4 – 10 – 110	3754	3770	3793
100 – 5 – 12 – 100	4128	4134	4178
120 – 5 – 12 – 120	5865	5861	5869
150 – 6 – 14 – 125	7220	7125	7182
200 – 8 – 20 – 100	8811	8755	8816

Table 5 Percentage (mean of ten instances) of forbidden indices when the rule in Proposition 5 is used

$(N - M - \ell)$	Mean % of forbidden indices
25 – 2 – 3	18%
50 – 3 – 6	12%
90 – 4 – 10	9%
120 – 5 – 12	8%
200 – 8 – 20	7%

8 Conclusion and perspectives

In this article, a scheduling problem on unrelated parallel machines with additional constraints, such as auxiliary resources and machine- and sequence-dependent setup times, and two objective functions is considered. This problem comes from the photolithography workshop of semiconductor manufacturing plants. A memetic algorithm is proposed. Its coding structure has the advantage of representing a dominant subset of solutions for both objective functions. The local search method, with its neighborhood operators, ensures that an optimal solution can be reached with the algorithm. Some properties justify the design of the algorithm and help to provide good solutions to large instances and improve our understanding of the problem.

The numerical experiments provide clues on how to set important parameters. One of our approaches is to analyze the impact of some other parameters of the algorithm and study its efficiency on other simplified problems in the literature, for instance without auxiliary resources. Also, an interesting question to solve consists in determining whether meaningful neighborhoods (as described in the paper) are connected, i.e., whether Theorem 2 remains true with the test introduced in Proposition 5. Finally, an important follow-up of this work is to adapt the algorithm to a bicriteria version of the problem.

References

- Akçali, E., Nemoto, K., & Uzsoy, R. (2001). Cycle time improvements for photolithography process in semiconductor manufacturing. *IEEE Transaction of Semiconductor Manufacturing*, 14(1), 48–56.
- Blazewicz, J., Lenstra, J. K., & Rinnooy Kan, A. H. (1983). Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics*, 5(1), 11–24.
- Cakici, E., & Mason, S. J. (2007). Parallel machine scheduling subject to auxiliary resource constraints. *Production Planning and Control*, 18(3), 217–225.
- Chandra, S. M., Mathirajan, M., Gopinath, R., & Sivakumar, A. I. (2008). Taboo search methods for scheduling a burn-in oven with non-identical job sizes and secondary resource constraints. *International Journal of Operational Research*, 3(1–2), 119–139.
- de Diaz, S. L. M., Fowler, J. W., Pfund, M. E., Mackulak, G. T., & Hickie, M. (2005). Evaluating the impacts of reticle requirements in semiconductor wafer fabrication. *IEEE Transactions on Semiconductor Manufacturing*, 18(4), 622–632.
- Kempf, K., Uzsoy, R., & Wang, C.-S. (1998). Scheduling a single batch processing machine with secondary resource constraints. *Journal of Manufacturing Systems*, 17(1), 37–51.
- Leung, J. Y. T., & Li, C. L. (2008). Scheduling with processing set restrictions: A survey. *International Journal of Production Economics*, 116(2), 251–262.
- Mönch, L. (2002). A genetic algorithm heuristic applied to stepper scheduling. In *Proceedings of the International Conference on Modelling and Analysis of Semiconductor Manufacturing*, Tempe, USA, (pp. 276–281).
- Mönch, L., Fowler, J. W., & Mason, S. J. (2012). *Production planning and control for semiconductor wafer fabrication facilities: Modeling, analysis, and systems*. New York: Springer.
- Mönch, L., Fowler, J. W., Dauzère-Pérès, S., Mason, S. J., & Rose, O. (2011). A survey of problems, solution techniques, and future challenges in scheduling semiconductor manufacturing operations. *Journal of Scheduling*, 14(6), 583–599.
- Sarin, S. C., Varadarajan, A., & Wang, L. (2011). A survey of dispatching rules for operational control in wafer fabrication. *Production Planning & Control*, 22(1), 4–24.